ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте

на тему: Круг: мобильное приложение для образовательной среды на базе RAG с ASR пайплайном

Выполнил:

Студент группы БКНАД222	подпись студента	Анастасия Ярославовна Голубкова
	Подпись	И.О.Фамилия

15.05.2026
Дата

Принял:

Руководитель проекта Вадим Олегович Грищенко
Имя, Отчество, Фамилия

Должность, ученое звание

ФКН НИУ ВШЭ
Место работы (Компания или подразделение НИУ ВШЭ)

Дата проверки 15.05 2026	10	подпись руководителя
	Оценка (по 10-ти бальной шкале)	Подпись

Москва 2026

Содержание

1 Введение **3**

2 Описание функциональных и нефункциональных требований к программному проекту **4**

3 Обзор литературы **6**
3.1 Существующие подходы к информационному поиску и генерации . . . 6
3.2 Подход retrieval-augmented generation . . . 7
3.3 Обоснование архитектуры RAG-сервиса . . . 7
3.4 Векторные базы данных . . . 7
3.5 Автоматическое распознавание речи . . . 8
3.6 Криптографические методы защиты . . . 8
3.7 Обоснование архитектурных решений системы . . . 8

4 Основная часть **8**
4.1 Краткий обзор архитектуры . . . 8
4.1.1 Назначение и границы системы; теоретическое обоснование декомпозиции . . . 8
4.1.2 Клиентский уровень . . . 9
4.1.3 Сервер приложений (ASP.NET Core) . . . 9
4.1.4 Данные и хранилища . . . 9
4.1.5 Внешние и вспомогательные сервисы . . . 10
4.2 RAG-сервис . . . 10
4.2.1 Назначение и архитектурная роль . . . 10
4.2.2 Общая модель обработки запроса . . . 10
4.2.3 Тематическая валидация (topic gate) . . . 11
4.2.4 Переформулировка запроса . . . 11
4.2.5 Извлечение контекста (retrieval) . . . 11
4.2.6 Генерация ответа . . . 12
4.2.7 Согласованность данных . . . 12
4.2.8 Контракт API и форматы данных . . . 13
4.2.9 Стек технологий и параметры моделей . . . 13
4.2.10 Интеграция с сервером приложений . . . 13
4.3 ASR-пайплайн . . . 13
4.3.1 Назначение . . . 13
4.3.2 Архитектурная модель . . . 16
4.3.3 Общая схема обработки . . . 16
4.3.4 Этапы обработки . . . 16
4.3.5 Очередь задач и фоновый обработчик . . . 16
4.3.6 Взаимодействие с ASR-провайдером . . . 17
4.3.7 Контроль качества и модерация . . . 17
4.3.8 Ошибки и отказоустойчивость . . . 17
4.3.9 Связь с векторной индексацией . . . 18
4.4 Шифрование сообщений . . . 18
4.4.1 Модель безопасности . . . 18
4.4.2 Криптографическая схема . . . 18
4.4.3 Свойства безопасности . . . 19
4.4.4 Роль сервера и хранилища . . . 19
4.4.5 Расшифровка на клиенте . . . 19
4.4.6 Управление ключевой парой пользователя . . . 20
4.4.7 Практические аспекты реализации . . . 20
4.5 Веб-сокеты для онлайн-чатов . . . 20
4.5.1 Обоснование выбора абстракции SignalR . . . 20
4.5.2 Аутентификация соединения . . . 21
4.5.3 Модель соединений и групп . . . 21
4.5.4 Событийная модель обмена . . . 21
4.5.5 Устойчивость соединения . . . 22

4.5.6 Интеграция с системой сообщений . . . 22
4.5.7 CORS и особенности разработки . . . 22
4.5.8 Преимущества выбранного подхода . . . 23
4.6 Асинхронность . . . 23
4.6.1 Асинхронность в ASP.NET Core . . . 23
4.6.2 Фоновые службы и очереди . . . 24
4.6.3 Изоляция процессов и контейнеризация . . . 24
4.6.4 Параллелизм и конкурентный доступ . . . 25
4.6.5 Синхронизация и блокировки . . . 25
4.6.6 Преимущества асинхронной архитектуры . . . 25
4.7 Пользовательский интерфейс (UI) . . . 25
4.7.1 Структура клиентского приложения . . . 25
4.7.2 Управление состоянием и реактивная модель . . . 26
4.7.3 Интеграция с транспортным уровнем . . . 26
4.7.4 Конфигурация окружения . . . 26
4.7.5 Обработка состояний и ошибок . . . 27
4.7.6 Согласованность с серверной логикой . . . 27
4.7.7 Преимущества архитектуры UI . . . 27
4.8 Тестирование . . . 27
4.8.1 Модульные и интеграционные тесты API . . . 28
4.8.2 Непрерывная интеграция и окружения . . . 28
4.8.3 Оценка качества RAG (офлайн) . . . 28
4.8.4 Нагрузочное тестирование канала SignalR . . . 30
4.8.5 Выводы по результатам тестирования . . . 31

Аннотация

Работа посвящена разработке мобильной программной системы, предназначенной для организации защищённого взаимодействия студентов и наставников, а также накопления и использования экспертных знаний в образовательной среде. Архитектура приложения сочетает клиент-серверную модель с подсистемой retrieval-augmented generation (RAG), обеспечивающей генерацию ответов на основе извлекаемого контекста из векторного хранилища интервью.

Формирование базы знаний осуществляется с использованием асинхронного ASR-пайплайна, преобразующего пользовательские аудиоматериалы в текст с последующей ручной верификацией перед индексированием. Обмен сообщениями реализован в режиме реального времени с использованием WebSocket (SignalR) и сопровождается прикладным шифрованием на стороне клиента.

В рамках работы выполнена реализация ключевых компонентов системы, а также проведено многоуровневое тестирование, включающее проверку API, оценку качества RAG-конвейера и нагрузочные испытания канала реального времени. Полученные результаты подтверждают работоспособность и масштабируемость предложенного решения.

1 Введение

В современных условиях цифровизации образования особую значимость приобретает эффективное взаимодействие между студентами и наставниками, обеспечивающее передачу практического опыта и поддержку профессионального развития. Традиционные средства коммуникации, как правило, не учитывают специфические требования образовательной среды, включая необходимость обеспечения конфиденциальности диалогов, систематизации накопленных знаний и быстрого доступа к ним.

Одним из перспективных направлений решения данной задачи является интеграция интеллектуальных методов обработки информации в коммуникационные платформы. В частности, использование больших языковых моделей позволяет автоматизировать ответы на типовые вопросы, однако их применение без дополнительного контроля может приводить к генерации недостоверной информации. В связи с этим актуальным является подход retrieval-augmented generation (RAG), предполагающий формирование ответа на основе явно извлекаемых фрагментов знаний.

Целью настоящей работы является разработка мобильного приложения, объединяющего защищённый канал связи между пользователями с интеллектуальной подсистемой поиска и генерации ответов, опирающейся на накопленную базу интервью и консультаций.

Архитектура разработанной системы строится на принципах модульности, безопасности и масштабируемости. В качестве базовой используется клиент-серверная модель, дополненная специализированными подсистемами. Ключевым элементом является RAG-сервис, реализующий гибридный механизм извлечения информации (лексический и семантический поиск) с последующей генерацией ответа языковой моделью на основе релевантного контекста.

Источником данных для RAG-сервиса служит база знаний, формируемая непосредственно пользователями. Загружаемые аудио- и видеоматериалы интервью обрабатываются с помощью ASR-пайплайна, реализованного как асинхронный серверный конвейер. Полученные транскрипции проходят этап ручной верификации, после чего сегментируются и индексируются в реляционном и векторном хранилищах. Такой подход обеспечивает контроль качества данных и исключает попадание некорректной информации в систему.

Для обеспечения конфиденциальности переписки реализована модель прикладного шифрования: сообщения шифруются на стороне клиента с использованием гибридной криптографической схемы и передаются через сервер исключительно в зашифрованном виде. Обмен сообщениями в реальном времени организован с использованием WebSocket (SignalR), что обеспечивает низкую задержку доставки и устойчивость соединения.

Дополнительным архитектурным аспектом является использование асинхронных механизмов обработки, позволяющих эффективно распределять нагрузку между компонентами системы. В частности, длительные операции (например, транскрибация аудио) выполняются в фоновом режиме с использованием очередей задач и отдельных обработчиков.

Для подтверждения корректности и эффективности разработанной системы проведён комплекс тестовых испытаний. Он включает модульное и интеграционное тестирование серверного API, офлайн-оценку качества RAG-пайплайна с использованием подхода LLM-as-judge, а также нагрузочные испытания канала обмена сообщениями. Полученные результаты демонстрируют устойчивость системы и соответствие заявленным требованиям.

Структура выпускной квалификационной работы отражает этапы проектирования и реализации системы. В первой главе рассматриваются теоретические основы используемых подходов, включая методы RAG, принципы построения систем распознавания речи и криптографические механизмы защиты данных. Во второй главе приводится архитектура системы и описание взаимодействия её компонентов. В третьей главе подробно рассматриваются особенности реализации ключевых подсистем: RAG-сервиса, ASR-пайплайна, механизма шифрования сообщений, обмена данными в реальном времени и клиентского приложения. Четвёртая глава посвящена вопросам асинхронной обработки и организации вычислительных процессов. В пятой главе описывается методология тестирования и анализируются полученные результаты. В заключении подводятся итоги работы, формулируются основные выводы и обозначаются направления дальнейшего развития системы.

2 Описание функциональных и нефункциональных требований к программному проекту

Подлежащий реализации программный комплекс относится к классу распределённых информационных систем с мобильным клиентским терминалом и многослойной серверной частью. Функциональная декомпозиция опирается на различие между *операционными* функциями (управление субъектами доступа и объектами предметной области) и *аналитическими* функциями (семантический поиск и генерация текста по извлечённому контексту). Ниже требования сформулированы в терминах наблюдаемого поведения системы и ограничений на её реализацию.

Функциональные требования Разрабатываемая система должна обеспечивать следующий функционал:

- **Идентификация и аутентификация пользователей.** Реализация подсистемы аутентификации с использованием токенов формата JWT, поддержка разграничения доступа на основе ролей (студент, ментор, администратор), а также проверка прав доступа при обращении к API и каналам реального времени.
- **ИИ-ассистент (RAG).** Обработка пользовательских запросов с использованием RAG-подсистемы, включающей:
  - формирование поискового запроса с учётом контекста диалога;
  - гибридный поиск (лексический и семантический);

  - генерацию ответа на основе извлечённых данных;
  - сохранение результатов в истории сообщений.

- **Обмен сообщениями в реальном времени.** Организация дуплексного взаимодействия между пользователями с доставкой сообщений в режиме, близком к реальному времени, с использованием WebSocket через абстракцию SignalR.

- **Шифрование сообщений.** Обеспечение прикладного шифрования сообщений на стороне клиента с передачей и хранением на сервере исключительно зашифрованных данных.

- **Загрузка и обработка медиа.** Поддержка загрузки аудио- и видеофайлов интервью через HTTP API с контролем допустимого размера и формата данных.

- **ASR-пайплайн.** Реализация асинхронного конвейера распознавания речи, включающего:
  - постановку задачи в очередь;
  - вызов внешнего ASR-сервиса;
  - получение транскрипции;
  - сохранение результата в базе данных.

- **Организация виртуального рабочего пространства.** Предоставление пользователям (студентам и менторам) возможности формирования совместного рабочего пространства в рамках установленного взаимодействия. Рабочее пространство должно обеспечивать:
  - управление участниками и их ролями;
  - планирование встреч (созвоны) с указанием даты, времени и внешних ссылок;
  - постановку задач с назначением ответственных исполнителей;
  - установку сроков выполнения (дедлайнов);
  - отслеживание статусов задач (на проверке, в работе, выполнена);
  - ведение комментариев и обсуждений в рамках задач;
  - хранение истории изменений состояния задач.

- **Работа с корпусом знаний.** Предоставление пользователю возможности просмотра интервью, прошедших модерацию, включая текстовое содержимое и метаданные (идентификаторы, временные метки, принадлежность к диалогу).

- **Модерация данных.** Наличие этапа ручной или полуавтоматической проверки транскрипций перед включением в корпус знаний.

- **Индексирование данных.** Разбиение текста на чанки, построение эмбеддингов и загрузка данных в векторное хранилище с последующим использованием в RAG-поиске.

- **Отображение состояния на клиенте.** Отображение пользователю состояния соединения, статусов операций, расшифрованных сообщений и связанных сущностей в соответствии с контрактом API.

- **Функциональность тестирования системы.** Система должна обеспечивать возможность проверки корректности работы отдельных компонентов и системы в целом. В частности, должны быть реализованы:
  - модульные тесты бизнес-логики и вспомогательных функций;
  - интеграционные тесты HTTP API с использованием тестового окружения;
  - проверки корректности обработки пользовательских сценариев (регистрация, работа с чатами, загрузка данных);
  - возможность выполнения тестов без подключения к внешним сервисам.

- **Тестирование RAG-подсистемы.** Система должна обеспечивать возможность оценки качества работы RAG-конвейера на различных этапах обработки запроса. В частности:

  - оценка качества извлечения релевантных фрагментов (retrieval);
  - оценка качества генерации ответов (generation);
  - поддержка офлайн-тестирования на фиксированном наборе запросов и эталонных ответов;
  - возможность использования внешних моделей (LLM-as-judge) для автоматизированной оценки результатов.

- **Нагрузочное тестирование.** Система должна обеспечивать возможность проведения нагрузочных испытаний для оценки производительности и устойчивости. В частности:
  - тестирование канала обмена сообщениями в реальном времени (SignalR);
  - измерение задержек доставки сообщений;
  - оценка пропускной способности при одновременной работе множества клиентов;
  - проверка устойчивости системы при длительных соединениях и сетевых сбоях.

Нефункциональные требования К системе предъявляются следующие нефункциональные требования:

- **Производительность и масштабируемость.** Система должна обеспечивать обработку пользовательских запросов с приемлемой задержкой, определяемой временем сетевого взаимодействия, поиска в векторном хранилище и генерации ответа. Архитектура должна поддерживать горизонтальное масштабирование за счёт увеличения числа экземпляров RAG-сервиса, фоновых воркеров и серверов приложений.

- **Доступность и отказоустойчивость.** Система должна сохранять работоспособность при частичных отказах внешних сервисов (ASR, LLM). Ошибки обработки не должны приводить к нарушению целостности данных; допускается отложенное выполнение задач и повторные попытки.

- **Безопасность.** Все сетевые взаимодействия должны осуществляться с использованием защищённого канала (TLS). Доступ к API и WebSocket-соединениям возможен только при наличии валидного JWT-токена. Сообщения пользователей должны храниться в зашифрованном виде, исключающем их восстановление на стороне сервера.

- **Согласованность данных.** Реляционная база данных является источником истины для пользовательских данных и статусов обработки. Векторное хранилище поддерживает согласованность по модели *eventual consistency*, обновляясь асинхронно по мере завершения индексации.

- **Надёжность обработки.** Каждая задача (например, транскрибация или индексирование) должна выполняться не более одного раза и иметь фиксированное состояние жизненного цикла (pending, processing, completed, failed).

- **Удобство сопровождения.** Система должна быть разделена на независимые компоненты (клиент, API, RAG-сервис, фоновые процессы), взаимодействующие через чётко определённые интерфейсы. Конфигурация внешних сервисов и секретов должна быть вынесена в параметры окружения.

- **Верифицируемость.** Должна быть обеспечена возможность автоматизированного тестирования на различных уровнях: модульном, интеграционном и системном. Для RAG-сервиса должны быть предусмотрены тесты как в изолированном режиме, так и при взаимодействии с реальными компонентами.

3 Обзор литературы

3.1 Существующие подходы к информационному поиску и генерации

Задача поиска и обработки текстовой информации является одной из центральных в современных интеллектуальных системах. Классические методы информационного поиска основываются на вероятностной модели релевантности, в рамках которой документы ранжируются по степени соответствия запросу пользователя. Одним из наиболее распространённых алгоритмов является BM25, подробно описанный в работе [16]. Данный подход обеспечивает высокую эффективность при лексическом поиске и широко применяется в промышленных системах.

С развитием нейронных методов появились модели семантического поиска, основанные на плотных векторных представлениях текста. В работе [6] предложен метод Dense Passage Retrieval (DPR), позволяющий находить релевантные документы на основе близости эмбеддингов в векторном пространстве. Дополнительно методы переранжирования с использованием трансформеров (например, [13]) позволяют повысить качество ранжирования за счёт более глубокого анализа контекста.

Современные системы поиска часто используют гибридные подходы, сочетающие лексический и семантический поиск, что подтверждается результатами исследований [8] и бенчмарка BEIR [18]. Такой подход позволяет объединить преимущества обоих методов: скорость и интерпретируемость BM25 с гибкостью нейросетевых моделей.

3.2 Подход retrieval-augmented generation

С появлением больших языковых моделей (LLM) возникла проблема генерации недостоверной информации (hallucination). В работе [7] предложена архитектура retrieval-augmented generation (RAG), в которой генерация ответа осуществляется с опорой на извлекаемые документы. Дальнейшие исследования [5, 17] показали, что использование retrieval существенно снижает уровень галлюцинаций и повышает достоверность ответов.

Дополнительным направлением является оптимизация формулировки запроса. Работы [10, 4] демонстрируют, что переформулировка запроса с учётом контекста улучшает качество извлечения и, как следствие, итоговой генерации.

В контексте оценки качества RAG-систем активно используется подход LLM-as-judge, предложенный в [20, 9]. Он позволяет автоматически оценивать релевантность и корректность ответов без участия человека, что особенно важно при разработке и сравнении моделей.

3.3 Обоснование архитектуры RAG-сервиса

В разработанной системе реализован гибридный RAG-пайплайн, сочетающий лексический поиск (BM25) и векторный поиск по эмбеддингам. Такой выбор обусловлен следующими факторами:

- BM25 обеспечивает быстрый отбор кандидатов и устойчивость к шуму в данных [16];
- векторный поиск позволяет учитывать семантическое сходство [6];
- объединение результатов (fusion) соответствует современным практикам гибридного поиска [18].

При этом в реализации BM25 используется in-memory индекс, формируемый на основе данных векторного хранилища. Такое решение соответствует принципу разделения источника истины и производных представлений: Qdrant выступает основным хранилищем, а BM25 — вычисляемым индексом.

Выбор отдельного микросервиса для RAG обусловлен высокой вычислительной сложностью операций генерации и необходимостью независимого масштабирования, что согласуется с современными архитектурными подходами к построению распределённых систем.

3.4 Векторные базы данных

Для хранения эмбеддингов используется специализированная векторная СУБД Qdrant [14], предназначенная для эффективного поиска ближайших соседей в высокоразмерных пространствах. В отличие от традиционных реляционных баз данных, такие системы оптимизированы под операции similarity search и обеспечивают высокую производительность при работе с большими объёмами текстовых данных.

Использование отдельного векторного хранилища позволяет:

- масштабировать поиск независимо от транзакционной нагрузки;
- хранить дополнительные метаданные для каждого чанка;
- реализовать гибридные схемы поиска.

3.5 Автоматическое распознавание речи

Для преобразования аудио в текст используется внешний ASR-сервис, интеграция с которым реализована через API [2, 1]. Подобные системы обеспечивают высокое качество распознавания за счёт использования обученных нейросетевых моделей.

Однако автоматическое распознавание не гарантирует полной точности, особенно в условиях шумов или специализированной терминологии. В связи с этим в разработанной системе введён этап ручной верификации, что соответствует практике разделения:

- сырого результата распознавания;
- опубликованного корпуса знаний.

Такой подход обеспечивает высокое качество данных, используемых в RAG-пайплайне.

3.6 Криптографические методы защиты

Для обеспечения конфиденциальности сообщений используется гибридная криптографическая схема, сочетающая симметричное и асимметричное шифрование. Алгоритм RSA [15] применяется для распределения ключей, а AES [3] — для шифрования содержимого сообщений.

Данный подход соответствует рекомендациям [12] и широко используется в системах защищённого обмена сообщениями. Применение шифрования на стороне клиента позволяет реализовать модель, близкую к end-to-end, при которой сервер не имеет доступа к открытому тексту.

3.7 Обоснование архитектурных решений системы

Анализ существующих подходов позволяет сформулировать обоснование ключевых архитектурных решений:

- использование RAG обусловлено необходимостью повышения достоверности ответов языковой модели;
- гибридный поиск выбран как компромисс между скоростью и качеством;
- разделение на микросервисы (API, RAG, ASR) обеспечивает масштабируемость и отказоустойчивость;
- применение векторной базы данных позволяет эффективно работать с семантическим поиском;
- асинхронная обработка задач соответствует требованиям к системам с длительными операциями;
- клиентское шифрование обеспечивает высокий уровень безопасности данных.

Таким образом, разработанная архитектура опирается на современные научные и инженерные подходы, что подтверждается приведёнными источниками и соответствует текущему состоянию области.

4 Основная часть

Далее излагается конструктивное описание реализации с указанием обоснований архитектурных решений. Представление организовано в соответствии с физической декомпозицией репозитория: клиентское приложение (Ionic, Angular), сервер приложений (ASP.NET Core, Kestrel), реляционная СУБД PostgreSQL (образ с расширением PostGIS в файле оркестрации), векторная СУБД Qdrant, микросервис RAG (FastAPI), асинхронный индексатор (consumer) и внешние программные интерфейсы OpenAI и AssemblyAI.

4.1 Краткий обзор архитектуры

4.1.1 Назначение и границы системы; теоретическое обоснование декомпозиции

Система разбивается на три слабо связанных контура по критерию доминирующей нефункциональной характеристики.

Операционный контур реализует CRUD-операции над сущностями предметной области и инварианты целостности, выражаемые реляционной схемой. Обоснование: классические транзакционные СУБД обеспечивают свойства ACID и декларативные ограничения, что не воспроизводится векторными хранилищами без существенных компромиссов.

Контур медиа-обработки инкапсулирует задачи с переменной длительностью и внешней зависимостью (ASR). Обоснование: согласно модели *bounded context* предметно-ориентированного проектирования, длительные и недетерминированные процессы целесообразно изолировать от синхронного граничного контекста веб-API, чтобы не исчерпывать пул потоков и не нарушать SLA на время ответа приёмного эндпоинта.

Контур интеллектуального поиска объединяет векторизацию текста, поиск ближайших соседей и вызов LLM. Обоснование: данный контур демонстрирует иные профили нагрузки (CPU/GPU-интенсивные или сетевые задержки к облачному LLM) и иной цикл развёртывания (интерпретируемый стек Python), следовательно, его выделение в отдельный адресуемый сервис соответствует принципу независимого масштабирования и снижает связность кодовой базы.

Граница консистентности между PostgreSQL и Qdrant намеренно ослаблена до eventual consistency: векторный индекс является производным представлением реляционных данных; его обновление выполняется после явного перехода интервью в состояние, допускающее публикацию. Такой подход согласуется с теоремой CAP в постановке, где для подсистемы поиска выбирается устойчивость к разделению и доступность за счёт допустимой временной несогласованности с основным хранилищем.

4.1.2 Клиентский уровень

Клиент реализован как одностраничное приложение на платформе Ionic поверх Angular. С теоретической точки зрения данный выбор обеспечивает единый кодовый базис для гибридной мобильной сборки и веб-оболочки, что снижает стоимость сопровождения по сравнению с нативными реализациями под отдельные платформы при сохранении доступа к Web Crypto API в средах, где это требуется для клиентской криптографии.

Все сценарии, за исключением низкоуровневой доставки push-событий, используют протокол HTTP/1.1 или HTTP/2 поверх TLS к REST-контроллерам; для чатов устанавливается долгоживущее соединение SignalR. Криптографическая обработка текста сообщения выполняется исключительно на клиенте до сериализации тела запроса или вызова метода Hub, что формализует модель угроз “честный, но любопытный оператор сервера”.

4.1.3 Сервер приложений (ASP.NET Core)

Сервер приложений построен на конвейере middleware ASP.NET Core. Аутентификация реализована по схеме Bearer JWT: параметры валидации (издатель, аудитория, ключ подписи, контроль срока действия) конфигурируются секцией `Jwt`. Для SignalR зарегистрировано событие `OnMessageReceived`, извлекающее токен из строки запроса при обращении к пути `/chathub`, поскольку спецификация WebSocket в браузерной среде ограничивает установку произвольных заголовков `Authorization` на этапе рукопожатия.

Ограничение `MaxRequestBodySize` на уровне Kestrel и `MultipartBodyLengthLimit` на уровне `FormOptions` согласованы между собой и задаются порядка 200 MiB, что оправдано приёму несжатого или слабо сжатого аудио. Доступ к данным осуществляется через `DbContext` (Entity Framework Core) с провайдером Npgsql; в средах разработки и Docker при старте приложения выполняется миграция схемы и идемпотентные DDL-операции для столбцов, связанных с политикой RAG-чата.

Политика CORS объявлена явным перечислением доверенных `origin` и флагом `AllowCredentials`, что необходимо для сценариев с кросс-доменной передачей cookie или заголовков авторизации в браузерной части клиента.

4.1.4 Данные и хранилища

Реляционная база хранит нормализованные отношения пользователей, чатов, сообщений и интервью. Поля сообщения включают шифротекст, два варианта зашифрованного симметричного ключа и вектор инициализации; открытый текст в таблице не сохраняется. Для интервью используются булевы и перечислимые признаки публикации и состояния очереди индексации (`check`, `in_queue` в терминологии реализации), что задаёт предикат отбора строк для consumer’a.

Векторная СУБД Qdrant хранит точки с плавающим вектором фиксированной размерности и полезной нагрузкой (payload), содержащей извлекаемый текст и метаданные (в частности, идентификатор интервью). Согласование размерности вектора с выходом модели вложения является инвариантом конфигурации: несоответствие приводит к отказу при создании коллекции или при поиске.

4.1.5 Внешние и вспомогательные сервисы

Микросервис RAG и процесс consumer подключаются к PostgreSQL и Qdrant по внутренней сети контейнеров. Вызовы OpenAI API инкапсулируются внутри RAG-сервиса; вызовы AssemblyAI — в фоновом сервисе транскрибации. Данная топология реализует *fault isolation*: отказ или деградация внешнего LLM не обязан приводить к остановке приёма HTTP-запросов основного API при условии корректной обработки ошибок и таймаутов на стороне вызывающего кода.

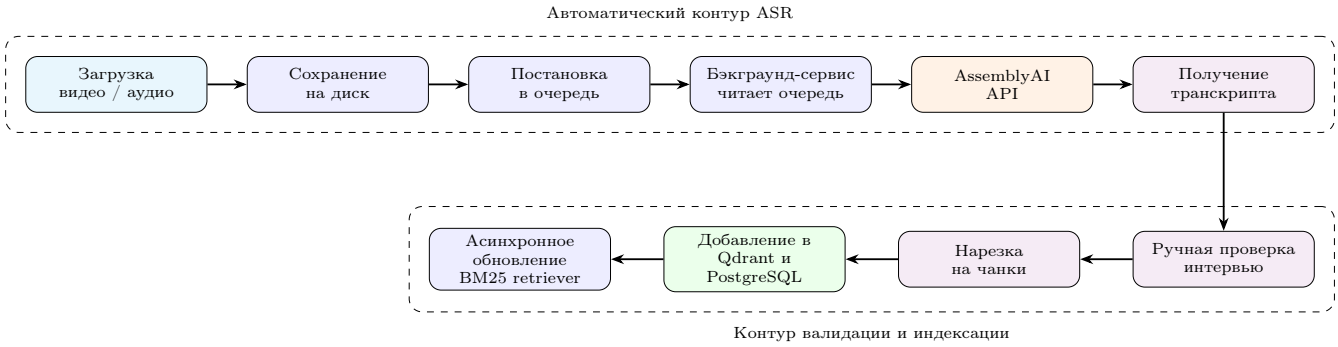

Рис. 1: Схема ASR-пайплайна: медиа загружается и асинхронно транскрибируется, после чего проходит ручную проверку и индексацию в поисковых хранилищах.

4.2 RAG-сервис

4.2.1 Назначение и архитектурная роль

RAG-сервис (retrieval-augmented generation) представляет собой специализированный программный компонент, реализующий генерацию ответов с использованием внешнего источника знаний. В отличие от классического применения больших языковых моделей, в котором ответ формируется исключительно на основе параметров модели, данный подход обеспечивает явную опору на извлекаемый контекст, что повышает точность и интерпретируемость результатов.

С архитектурной точки зрения RAG-сервис выделен в отдельный микросервис, взаимодействующий с основным сервером приложений по сетевому API. Такое решение обусловлено следующими факторами:

- высокая вычислительная сложность операций генерации и построения эмбеддингов;
- необходимость независимого масштабирования компонентов системы;
- различие используемых технологий (Python/FastAPI для RAG и ASP.NET Core для основного сервера);
- изоляция логики интеллектуальной обработки текста от бизнес-логики приложения.

Декомпозиция системы на независимые сервисы снижает связанность компонентов и обеспечивает гибкость при модификации отдельных частей системы.

4.2.2 Общая модель обработки запроса

Обработка пользовательского запроса реализуется как последовательность этапов, каждый из которых выполняет строго определённую функцию. Такая организация соответствует конвейерной модели вычислений и позволяет контролировать как качество результата, так и вычислительные затраты.

Пусть q — текущий запрос пользователя, а H — история диалога. Тогда полный процесс обработки можно представить в виде последовательного преобразования:

$$(H, q) \to q' \to \{D_1, \ldots, D_k\} \to a,$$

где q' — поисковая репрезентация запроса, D_i — извлечённые фрагменты знаний, а a — итоговый ответ.

4.2.3 Тематическая валидация (topic gate)

Первым этапом обработки является тематическая валидация запроса. На данном этапе языковая модель используется в режиме бинарной классификации для определения принадлежности запроса к предметной области системы.

С формальной точки зрения данный этап реализует правило ранней остановки вычислений: если вероятность релевантности запроса ниже заданного порога, дальнейшие ресурсоёмкие операции (вычисление эмбеддингов, обращение к индексам) не выполняются.

При отрицательном результате пользователю возвращается отказ в обработке запроса, сопровождаемый информацией о количестве оставшихся допустимых нерелевантных обращений. Такая политика позволяет ограничить злоупотребление системой и снизить нагрузку.

В случае ошибок формата ответа классификатора применяется консервативная стратегия: запрос считается релевантным, что обеспечивает доступность сервиса ценой возможного увеличения вычислительных затрат.

4.2.4 Переформулировка запроса

Следующим этапом является преобразование исходного запроса (H, q) в поисковую форму q'. Данный этап направлен на устранение проблемы лексического несовпадения между запросом и документами базы знаний.

Языковая модель формирует компактное представление запроса, учитывающее контекст диалога, включая ранее заданные вопросы и ответы. Это особенно важно для многотуровых диалогов, в которых отдельный запрос может быть семантически неполным без учёта истории.

4.2.5 Извлечение контекста (retrieval)

Извлечение релевантных данных реализуется как гибридный процесс, сочетающий лексический и семантический поиск. В отличие от классических архитектур, в которых этапы выполняются последовательно, в рассматриваемой системе оба механизма работают параллельно с последующим объединением результатов.

Лексический поиск (BM25) Лексический поиск реализован с использованием алгоритма BM25, однако он не представлен в виде отдельной базы данных или внешнего сервиса. Вместо этого в процессе выполнения RAG-сервиса поддерживается in-memory индекс, содержащий снимок корпуса документов.

Индекс представлен структурой `BM25Snapshot`, включающей:

- объект `BM25Okapi` из библиотеки `rank-bm25`;
- список текстовых чанков;
- соответствующие метаданные.

Формирование индекса осуществляется асинхронно на основе данных, хранящихся в векторном хранилище Qdrant. При обновлении индекса:

- выполняется последовательное чтение (scroll) всех элементов коллекции;
- извлекаются тексты и метаданные из payload;
- в отдельном потоке формируется новый экземпляр BM25;
- после завершения построения индекс атомарно подменяется.

Таким образом, BM25-индекс является производным представлением данных Qdrant и не выступает в роли независимого источника истины.

Поиск выполняется в отдельном потоке выполнения: запрос токенизируется, вычисляются оценки релевантности и выбирается множество наиболее подходящих документов.

Семантический поиск Параллельно с лексическим поиском выполняется семантический поиск по векторному индексу Qdrant. Для этого:

- вычисляется эмбеддинг запроса;
- выполняется поиск ближайших соседей по косинусной метрике;
- возвращается множество кандидатов с оценками сходства.

Данный этап позволяет выявить семантически близкие фрагменты, даже при отсутствии прямого лексического совпадения.

Объединение результатов Результаты лексического и семантического поиска объединяются с использованием метода reciprocal rank fusion (RRF). Данный подход учитывает позиции документов в каждом из ранжированных списков и формирует итоговое ранжирование без необходимости нормализации исходных оценок.

После объединения:

- выполняется фильтрация по порогу релевантности (для векторного поиска);
- устраняются дубликаты;
- формируется итоговый набор чанков.

Полученное множество документов используется для формирования контекста языковой модели.

Особенности реализации Следует отметить, что в рассматриваемой системе отсутствует жёсткое разделение этапов BM25, затем reranking». Вместо этого реализована параллельная схема извлечения с последующим объединением результатов, что отражает фактическую реализацию в коде и обеспечивает более устойчивое качество поиска.

4.2.6 Генерация ответа

На завершающем этапе языковая модель формирует ответ a на основе переданного контекста. Генерация осуществляется при фиксированном системном промпте, который ограничивает модель требованием использовать исключительно предоставленные данные.

Это позволяет:

- снизить вероятность генерации недостоверной информации;
- обеспечить воспроизводимость результатов;
- сформировать ссылки на конкретные источники (чанки).

История диалога H используется для поддержания связности ответа, тогда как фактическая обоснованность обеспечивается контекстом $\{D_i\}$.

4.2.7 Согласованность данных

Векторное хранилище (Qdrant) рассматривается как производное представление основной реляционной базы данных. Обновление индекса выполняется асинхронно, что соответствует модели eventual consistency.

Такой подход позволяет:

- не блокировать операции записи;
- масштабировать процесс индексации независимо;
- поддерживать высокую пропускную способность системы.

4.2.8 Контракт API и форматы данных

Взаимодействие с RAG-сервисом реализовано через HTTP API с использованием JSON. На вход передаются:

- текст запроса пользователя;
- история диалога в ролевой форме.

На выходе возвращаются:

- сгенерированный ответ;
- список идентификаторов источников;
- признак тематической релевантности.

Данный контракт является минимально достаточным и не раскрывает внутреннюю структуру реализации, что упрощает интеграцию и тестирование.

4.2.9 Стек технологий и параметры моделей

Для построения эмбеддингов используется модель семейства `text-embedding-3-small`, отображающая текст в пространство $\mathbb{R}^d$. Размерность d задаётся конфигурацией и должна совпадать с параметрами векторного индекса.

Для генерации и вспомогательных задач используется чат-модель (например, `gpt-4o-mini`) в трёх режимах:

- классификация темы;
- переформулировка запроса;
- генерация ответа.

Параметры модели (температура, ограничение длины ответа) подбираются таким образом, чтобы минимизировать вариативность и обеспечить предсказуемое время отклика.

4.2.10 Интеграция с сервером приложений

RAG-сервис интегрируется с сервером приложений ASP.NET Core как внешняя зависимость. Запрос формируется асинхронно, а полученный ответ сохраняется в историю сообщений.

При этом:

- семантическая обработка текста выполняется независимо;
- бизнес-логика (ограничения, политика доступа) реализуется на уровне основного сервера.

Такое разделение позволяет модифицировать продуктовые правила без изменения внутренней логики RAG-пайплайна.

4.3 ASR-пайплайн

4.3.1 Назначение

ASR-подсистема (automatic speech recognition) предназначена для преобразования аудио- и видеоматериалов интервью в текстовую форму, пригодную для последующего анализа, хранения и индексирования. Полученные транскрипции используются в качестве основного корпуса знаний для RAG-сервиса, что делает точность и полноту распознавания критически важными характеристиками системы.

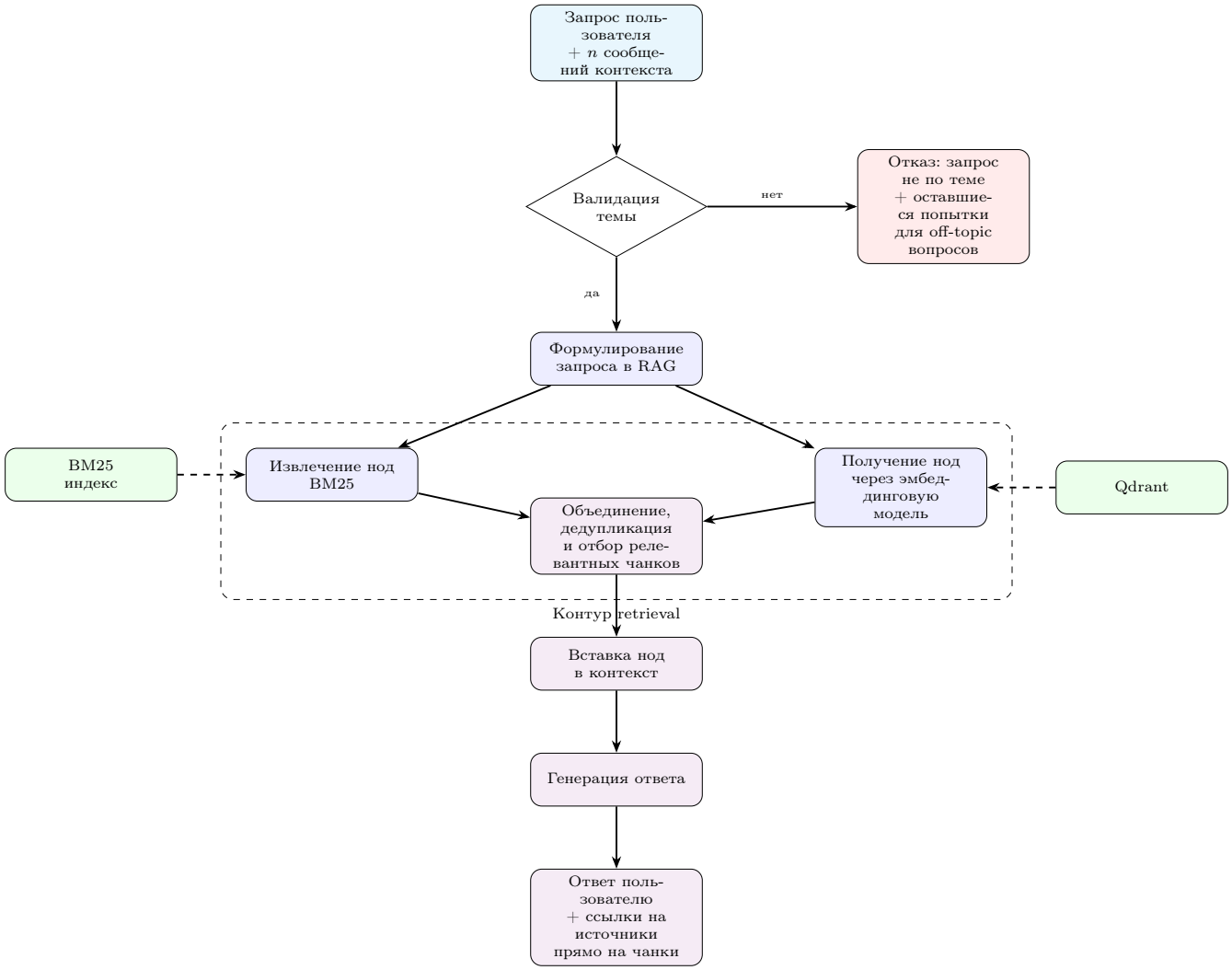

Рис. 2: Схема RAG-пайплайна: при нерелевантной теме пользователю возвращается отказ и число оставшихся попыток; при успешной валидации выполняется retrieval и генерация ответа с ссылками на чанки.

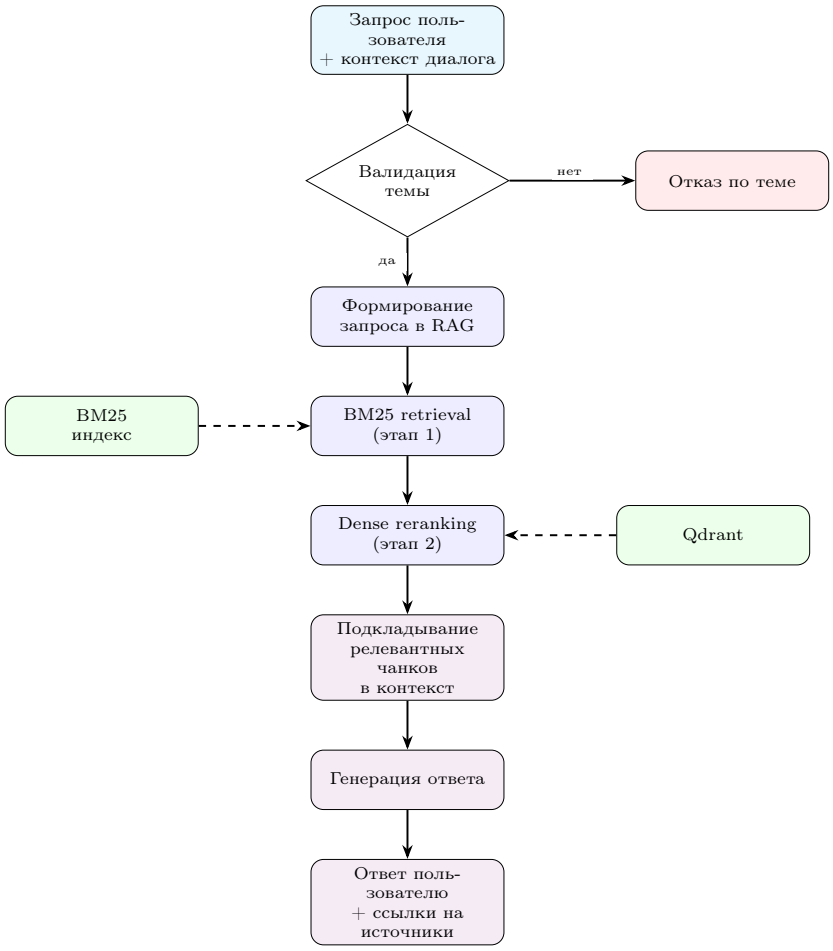

Рис. 3: RAG-конвейер: запрос пользователя и контекст диалога проходят тематическую валидацию, затем формируется поисковый запрос, выполняется двухэтапное извлечение релевантных чанков, после чего они подаются в контекст языковой модели для генерации ответа с указанием источников.

4.3.2 Архитектурная модель

Обработка реализована в виде асинхронного конвейера, разделённого на независимые этапы. Такой подход обусловлен высокой вариативностью времени выполнения операций распознавания речи, которое зависит от длительности аудиофайла, загрузки внешнего сервиса и сетевых задержек.

Использование синхронной модели (обработка в рамках HTTP-запроса) привело бы к:

- блокировке серверных потоков;
- увеличению времени отклика;
- снижению пропускной способности системы.

Поэтому применяется архитектурный паттерн producer–consumer, в котором пользовательский запрос лишь инициирует задачу, а фактическая обработка выполняется фоновым воркером.

4.3.3 Общая схема обработки

Пусть f — загруженный медиафайл. Тогда процесс преобразования можно формально представить как последовательность:

$$f \to T_{\text{raw}} \to T_{\text{validated}} \to \{D_1, \ldots, D_k\},$$

где T_{raw} — сырой транскрипт, $T_{\text{validated}}$ — отредактированный текст, а D_i — итоговые чанки, используемые в RAG.

4.3.4 Этапы обработки

Конвейер включает следующие стадии:

1. **Загрузка медиафайла.** Пользователь передаёт аудио- или видеофайл через HTTP-запрос.
2. **Сохранение на сервере.** Файл сохраняется в файловой системе или объектном хранилище с генерацией уникального идентификатора.
3. **Постановка в очередь.** В базу данных добавляется запись о задаче транскрибации, содержащая путь к файлу и статус обработки.
4. **Фоновая обработка.** Отдельный воркер извлекает задачи из очереди и инициирует процесс распознавания.
5. **Вызов внешнего ASR-сервиса.** Файл передаётся внешнему провайдеру распознавания речи.
6. **Получение транскрипции.** После завершения обработки сервис возвращает текстовый результат.
7. **Ручная верификация.** Пользователь или модератор проверяет корректность распознавания.
8. **Сегментация текста.** Транскрипция разбивается на чанки фиксированного или адаптивного размера.
9. **Индексирование.** Чанки сохраняются в реляционной базе данных и векторном индексе.
10. **Обновление BM25.** Лексический индекс перестраивается асинхронно на основе обновлённого корпуса.

4.3.5 Очередь задач и фоновый обработчик

Очередь задач реализует неблокирующий механизм передачи управления между пользовательским запросом и длительной обработкой. В качестве хранилища состояния используется база данных, в которой фиксируются:

- идентификатор задачи;
- статус выполнения;
- ссылка на исходный файл;
- временные метки.

Фоновый обработчик (consumer) выполняет следующие функции:

- выборка задач со статусом `pending`;
- обновление статуса на `processing`;
- выполнение транскрибации;
- сохранение результата;
- перевод задачи в финальное состояние (`completed` или `failed`).

Каждое задание обрабатывается в изолированном контексте зависимостей, что исключает гонки данных и повышает устойчивость системы.

4.3.6 Взаимодействие с ASR-провайдером

Интеграция с внешним сервисом распознавания речи реализована по трёхфазной схеме:

1. загрузка медиафайла;
2. запуск асинхронной транскрибации;
3. периодический опрос статуса выполнения.

Параметры взаимодействия (таймауты, интервалы опроса) задаются конфигурацией. Это позволяет адаптировать систему к различным условиям эксплуатации.

Результат обработки включает:

- текст транскрипции;
- временные метки;
- (при наличии) разметку говорящих.

Полученные данные преобразуются в унифицированный формат, используемый в системе.

4.3.7 Контроль качества и модерация

Ключевым элементом пайплайна является этап ручной проверки. Он выполняет функцию фильтрации ошибок распознавания и обеспечивает соответствие данных требованиям системы.

Разделение на:

- сырой транскрипт;
- валидированный текст

позволяет избежать попадания некорректной информации в индекс.

4.3.8 Ошибки и отказоустойчивость

Система предусматривает обработку следующих типов ошибок:

- сетевые ошибки при обращении к ASR-сервису;
- ошибки формата ответа;
- ошибки доступа (некорректный API-ключ).

В случае ошибки:

- задача переводится в состояние `failed`;
- информация записывается в журнал;
- допускается повторная попытка выполнения.

Особое внимание уделяется валидации API-ключа: наличие недопустимых символов или его отсутствие приводит к немедленному отказу от выполнения запроса.

4.3.9 Связь с векторной индексацией

После завершения транскрибации запускается этап подготовки данных для RAG-сервиса. Он включает:

- разбиение текста на чанки;
- вычисление эмбеддингов;
- пакетную запись в Qdrant;
- сохранение метаданных в PostgreSQL.

Данный процесс выполняется асинхронно, что позволяет:

- не блокировать основной пайплайн;
- обрабатывать большие объёмы данных;
- повторно инициировать индексацию при сбоях.

Таким образом, ASR-пайплайн формирует устойчивый и масштабируемый контур подготовки данных, обеспечивающий RAG-сервис качественным корпусом знаний.

4.4 Шифрование сообщений

4.4.1 Модель безопасности

В системе реализована защита сообщений на прикладном уровне, что позволяет обеспечить конфиденциальность передаваемых данных независимо от уровня доверия к серверной инфраструктуре. Предполагается, что сервер может быть скомпрометирован или находиться под контролем третьей стороны, однако при этом не имеет доступа к приватным ключам пользователей.

Данная модель соответствует классу угроз, в котором сервер рассматривается как *честный, но любопытный* участник (*honest-but-curious*). Он корректно выполняет протокол передачи сообщений, но потенциально способен анализировать доступные ему данные. В этих условиях криптографическая защита должна исключать возможность восстановления открытого текста сообщений даже при полном доступе к серверной базе данных.

Таким образом, система реализует end-to-end-подобную модель безопасности, при которой расшифрование возможно исключительно на стороне клиента.

4.4.2 Криптографическая схема

Для защиты сообщений используется гибридная криптографическая схема, сочетающая симметричное и асимметричное шифрование. Такой подход является стандартом де-факто для систем обмена сообщениями, поскольку позволяет объединить эффективность симметричных алгоритмов и гибкость распределения ключей асимметричных методов.

В системе применяются:

- симметричное шифрование сообщения алгоритмом AES в режиме GCM;
- асимметричное шифрование симметричного ключа алгоритмом RSA-OAEP.

Формально процесс шифрования можно описать следующим образом. Пусть M — исходное сообщение. Тогда:

1. генерируется случайный сеансовый ключ K из пространства $\{0,1\}^n$;
2. формируется случайный вектор инициализации IV;
3. вычисляется криптограмма:
$$C = \text{AES-GCM}_K(M, IV),$$
включающая зашифрованный текст и тег аутентификации;
4. ключ K шифруется публичными ключами получателей:
$$E_i = \text{RSA-OAEP}_{PK_i}(K),$$
где PK_i — публичный ключ i-го участника.

На сервер передаются $(C, IV, \{E_i\})$, при этом сам ключ K в открытом виде нигде не хранится.

4.4.3 Свойства безопасности

Используемая схема обеспечивает следующие свойства:

- **Конфиденциальность.** Открытый текст сообщения недоступен серверу, так как для его восстановления требуется приватный ключ пользователя.
- **Целостность и аутентичность.** Режим GCM обеспечивает проверку неизменности данных с использованием тега аутентификации. Любая модификация криптограммы приводит к ошибке расшифрования.
- **Независимость сообщений.** Каждое сообщение использует уникальный сеансовый ключ, что предотвращает компрометацию всей переписки при утечке одного ключа.
- **Защита от повторного использования.** Использование уникальных IV предотвращает атаки, связанные с повторным использованием ключа.

4.4.4 Роль сервера и хранилища

Сервер выполняет функции транспортного уровня и хранения данных. Его обязанности включают:

- валидацию структуры входящих сообщений;
- транзакционную запись криптографических артефактов;
- маршрутизацию сообщений между участниками диалога.

При этом сервер не выполняет операции расшифрования и не обладает доступом к ключевому материалу. В базе данных хранятся исключительно:

- зашифрованное сообщение C;
- параметры шифрования (например, IV);
- набор зашифрованных ключей $\{E_i\}$;
- метаданные (идентификаторы пользователей, временные метки).

Таким образом, компрометация серверной базы данных не приводит к раскрытию содержимого сообщений.

4.4.5 Расшифровка на клиенте

Процесс расшифрования выполняется исключительно на стороне клиента. Для каждого полученного сообщения клиент:

1. выбирает соответствующую криптограмму ключа E_i;
2. расшифровывает её с использованием своего приватного ключа:
$$K = \text{RSA-OAEP}^{-1}_{SK_i}(E_i);$$
3. выполняет расшифрование сообщения:
$$M = \text{AES-GCM}^{-1}_{K}(C, IV).$$

Ошибки на любом этапе (например, нарушение целостности или некорректный ключ) локализуются на уровне конкретного сообщения и не влияют на работоспособность канала связи в целом.

4.4.6 Управление ключевой парой пользователя

Каждый пользователь системы обладает собственной асимметричной ключевой парой. Регистрация публичного ключа выполняется через API сервера и сопровождается валидацией параметров (тип алгоритма, размер ключа, формат представления).

Публичный ключ хранится на сервере и используется для шифрования сеансовых ключей. Приватный ключ:

- генерируется на стороне клиента;
- сохраняется в защищённом хранилище (например, Web Crypto API или системный keystore);
- не передаётся на сервер.

Такой подход гарантирует, что компрометация серверной инфраструктуры не приводит к раскрытию переписки без одновременного компрометирования клиентских устройств.

4.4.7 Практические аспекты реализации

В реализации учитываются ограничения протокола HTTP и криптографических библиотек. В частности:

- API-ключи передаются в заголовках и валидируются на допустимость символов;
- ошибки шифрования или расшифрования фиксируются и логируются;
- операции криптографии выполняются с использованием стандартных библиотек платформы (например, Web Crypto API).

Дополнительно система проектируется с учётом возможного расширения, включая:

- поддержку группового шифрования;
- ротацию ключей;
- переход на более современные схемы (например, эллиптические кривые).

Таким образом, реализованный механизм шифрования обеспечивает высокий уровень защиты данных при сохранении производительности и масштабируемости системы.

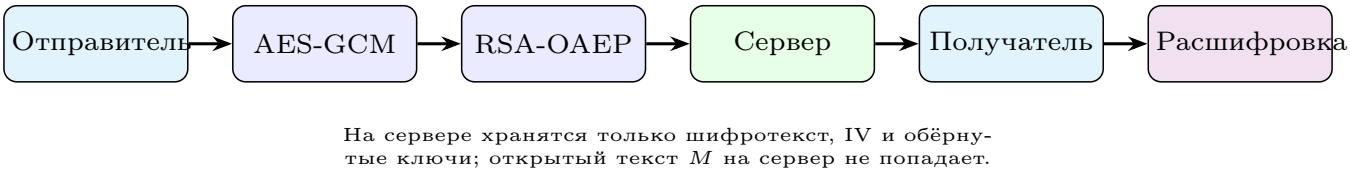

Рис. 4: Схема прикладного шифрования сообщений (гибрид AES-GCM + RSA-OAEP).

4.5 Веб-сокеты для онлайн-чатов

4.5.1 Обоснование выбора абстракции SignalR

Для реализации обмена сообщениями в режиме реального времени в системе используется протокол WebSocket (RFC 6455), обеспечивающий установление двустороннего канала связи поверх TCP. В отличие от классической модели HTTP-запрос–ответ, WebSocket позволяет серверу инициировать передачу данных без повторного запроса со стороны клиента, что существенно снижает задержки доставки сообщений.

Однако протокол WebSocket определяет лишь транспортный уровень и не регламентирует:

- механизм повторной доставки сообщений;
- управление состоянием соединения;
- маршрутизацию сообщений между участниками;

- вызов удалённых процедур на клиенте и сервере.

Для решения этих задач используется библиотека SignalR, реализующая прикладной уровень поверх WebSocket. SignalR предоставляет:

- абстракцию удалённых вызовов (remote procedure calls);
- управление подключениями и автоматическое восстановление соединения;
- механизм групповой маршрутизации сообщений;
- fallback-механизмы (Server-Sent Events, long polling).

С архитектурной точки зрения выбор SignalR соответствует принципу инверсии зависимостей: прикладной код оперирует событиями предметной области (сообщение, подключение, выход из чата), тогда как детали транспорта и протокола инкапсулируются фреймворком.

4.5.2 Аутентификация соединения

Установление WebSocket-соединения допускается только после успешной аутентификации пользователя. Для этого используется маркер доступа (JWT), выдаваемый сервером приложений.

В веб-сценарии передача токена осуществляется с учётом ограничений браузерной модели безопасности. На этапе рукопожатия WebSocket токен передаётся в query-параметрах или заголовках, после чего на стороне сервера выполняется:

- проверка криптографической подписи;
- валидация срока действия;
- сопоставление с контекстом пользователя.

При успешной аутентификации создаётся связка между соединением и идентификатором пользователя, которая используется в дальнейшем для маршрутизации сообщений.

4.5.3 Модель соединений и групп

В системе применяется модель групповой адресации сообщений. Каждая диалоговая сессия представляется в виде логической группы, в которую входят все участники чата.

Пусть G — множество участников диалога. Тогда группа определяется как:

$$G = \{u_1, u_2, \ldots, u_n\}.$$

Каждое WebSocket-соединение ассоциируется с конкретным пользователем и может входить в одну или несколько групп. Это позволяет:

- изолировать коммуникацию между различными чатами;
- минимизировать сетевой трафик за счёт адресной доставки;
- упростить управление доступом.

Добавление и удаление пользователя из группы выполняется динамически при входе и выходе из диалога.

4.5.4 Событийная модель обмена

Обмен сообщениями реализуется в событийной модели. Клиент и сервер взаимодействуют через именованные обработчики, что соответствует парадигме event-driven архитектуры.

Основные операции протокола:

- **Join** — подключение пользователя к диалоговой сессии;
- **SendMessage** — отправка сообщения в чат;
- **Leave** — выход пользователя из сессии;

- **ReceiveMessage** — серверное уведомление о новом сообщении.

Передача данных осуществляется в формате JSON, что обеспечивает переносимость и совместимость с различными клиентскими платформами.

Сервер выполняет:

- приём сообщений от клиентов;
- валидацию структуры данных;
- сохранение сообщений в базе данных;
- рассылку сообщений всем участникам группы.

4.5.5 Устойчивость соединения

В условиях нестабильных сетевых соединений (особенно на мобильных устройствах) важным требованием является автоматическое восстановление канала связи.

SignalR реализует механизм переподключения, при котором:

- клиент автоматически пытается восстановить соединение;
- после восстановления повторно выполняется процедура присоединения к группе;
- состояние диалога синхронизируется.

Это позволяет обеспечить непрерывность пользовательского опыта без ручного вмешательства.

4.5.6 Интеграция с системой сообщений

Подсистема WebSocket интегрируется с основной серверной логикой следующим образом:

- сообщения сохраняются в реляционной базе данных (PostgreSQL);
- WebSocket используется для доставки сообщений в реальном времени;
- история сообщений восстанавливается через HTTP API при подключении клиента.

Такое разделение обеспечивает:

- надёжное хранение данных;
- масштабируемость системы;
- возможность повторного воспроизведения истории диалога.

4.5.7 CORS и особенности разработки

При использовании веб-клиента (в частности, на базе Ionic) необходимо учитывать ограничения политики междоменного взаимодействия (CORS). Для корректной работы WebSocket и HTTP-запросов требуется настройка доверенных источников.

В конфигурации сервера задаётся политика:

- перечисление разрешённых доменов;
- разрешение передачи учётных данных (`AllowCredentials`);
- указание допустимых методов и заголовков.

Нарушение этих требований приводит к блокировке запросов на уровне браузера, что делает корректную настройку CORS обязательной частью системы.

4.5.8 Преимущества выбранного подхода

Использование SignalR поверх WebSocket обеспечивает:

- низкую задержку доставки сообщений;
- устойчивость к сетевым сбоям;
- упрощение разработки за счёт абстракции транспортного уровня;
- масштабируемость при увеличении числа пользователей.

Таким образом, выбранная архитектура позволяет реализовать эффективную и надёжную систему обмена сообщениями в реальном времени.

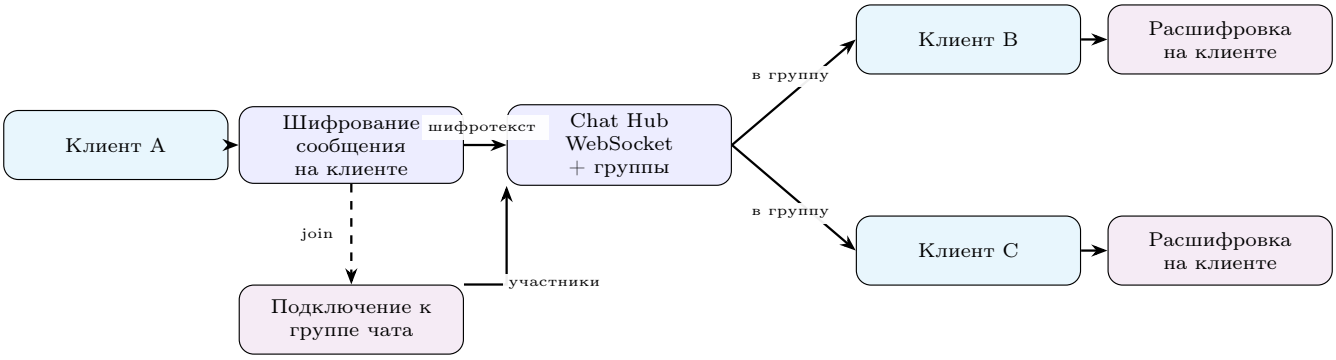

Рис. 5: Схема Chat Hub: сообщение шифруется на стороне клиента, передаётся через WebSocket в группу и рассылается участникам, которые расшифровывают его локально.

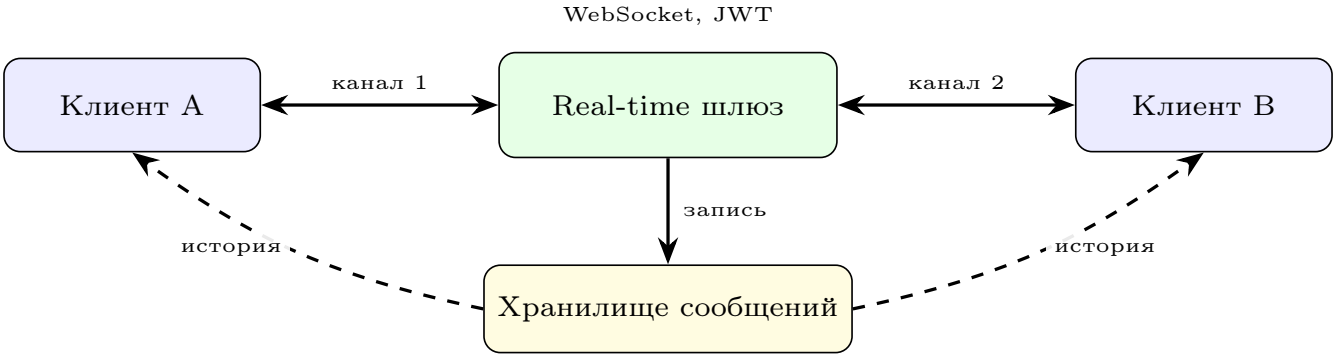

Рис. 6: Схема real-time обмена: клиенты поддерживают авторизованные соединения с серверным шлюзом, который доставляет события и сохраняет сообщения в постоянном хранилище.

4.6 Асинхронность

4.6.1 Асинхронность в ASP.NET Core

В серверной части системы используется асинхронная модель обработки запросов, реализуемая средствами платформы ASP.NET Core на основе механизма `async/await`. Данная модель ориентирована на неблокирующее выполнение операций ввода-вывода (I/O), таких как обращение к базе данных, файловой системе и внешним сервисам.

С точки зрения выполнения, асинхронный метод освобождает поток выполнения в момент ожидания результата операции и возвращает управление пулу потоков. Это позволяет одному и тому же набору серверных потоков обслуживать большее количество параллельных запросов.

Формально, если операция имеет длительность:

$$T = T_{\text{CPU}} + T_{\text{I/O}},$$

то асинхронная модель позволяет эффективно скрыть компоненту $T_{\text{I/O}}$, не удерживая поток в состоянии ожидания.

В рассматриваемой системе асинхронность применяется для:

- обработки HTTP-запросов;
- взаимодействия с базой данных (через асинхронные драйверы);
- вызова внешних API (ASR, RAG);
- записи и чтения файлов.

Это обеспечивает высокую пропускную способность сервера при смешанной нагрузке, характерной для приложений с интенсивным использованием сетевых ресурсов.

4.6.2 Фоновые службы и очереди

Для выполнения длительных операций (например, транскрибации аудио) используется разделение обработки на два независимых этапа:

- приём пользовательского запроса;
- асинхронная фоновая обработка.

Данный подход реализует классический паттерн *producer–consumer*. В рамках этого паттерна:

- HTTP-контроллер выступает в роли производителя задач;
- очередь задач выступает в роли буфера;
- фоновые воркеры выполняют роль потребителей.

После получения запроса задача помещается в очередь, а пользователю немедленно возвращается ответ о принятии задачи. Это позволяет:

- избежать блокировки клиентского запроса;
- перераспределить нагрузку во времени;
- масштабировать обработку независимо от количества входящих запросов.

Фоновая служба реализуется с использованием интерфейса `IHostedService` или аналогичного механизма и выполняется в отдельном контексте выполнения.

4.6.3 Изоляция процессов и контейнеризация

Компоненты системы, выполняющие ресурсоёмкие или специализированные задачи (например, RAG-сервис или ASR-consumer), размещаются в отдельных контейнерах Docker.

Такое разделение обеспечивает:

- изоляцию адресного пространства процессов;
- независимость зависимостей (разные языки и библиотеки);
- управление жизненным циклом через оркестратор;
- устойчивость к сбоям отдельных компонентов.

Например, отказ Python-процесса, выполняющего RAG или ASR, не приводит к остановке основного сервера приложений (Kestrel). Оркестратор контейнеров (Docker или Kubernetes) может автоматически перезапустить сбойный компонент без влияния на остальные части системы.

4.6.4 Параллелизм и конкурентный доступ

При наличии нескольких экземпляров фоновых обработчиков возникает задача координации доступа к общим данным. В частности, необходимо исключить ситуацию, при которой одна и та же задача обрабатывается несколькими воркерами одновременно.

Для решения данной проблемы используется конкурентно-безопасный механизм отбора задач, реализуемый на уровне базы данных. Типичная схема включает:

- выборку задач в состоянии `pending`;
- атомарное обновление статуса на `processing`;
- использование транзакций или блокировок уровня строки.

Это гарантирует, что каждая задача будет обработана ровно одним воркером.

4.6.5 Синхронизация и блокировки

В отдельных компонентах системы (например, при обновлении in-memory индексов) используется явная синхронизация с помощью примитивов блокировки. Это необходимо для обеспечения консистентности данных при параллельном доступе.

В частности, при обновлении лексического индекса:

- новый индекс строится в отдельном потоке;
- после завершения выполняется атомарная подмена структуры;
- доступ к текущему состоянию защищён блокировкой (`lock`).

Такой подход минимизирует время удержания блокировки и снижает влияние синхронизации на производительность системы.

4.6.6 Преимущества асинхронной архитектуры

Использование асинхронности и фоновых процессов обеспечивает следующие преимущества:

- эффективное использование вычислительных ресурсов;
- высокая пропускная способность при I/O-нагрузке;
- устойчивость к длительным операциям;
- возможность горизонтального масштабирования;
- снижение времени отклика пользовательских запросов.

Таким образом, асинхронная архитектура является ключевым элементом системы, обеспечивающим её производительность и масштабируемость в условиях реальной эксплуатации.

4.7 Пользовательский интерфейс (UI)

4.7.1 Структура клиентского приложения

Клиентская часть системы реализована с использованием компонентной архитектуры, характерной для современных одностраничных приложений. Архитектура разделена на три основных слоя:

- **Слой представления (presentation layer)** — отвечает за отображение пользовательского интерфейса и обработку пользовательских событий;
- **Слой управления состоянием (state management)** — хранит текущее состояние приложения и обеспечивает его согласованность;
- **Слой доступа к данным (data access layer)** — инкапсулирует взаимодействие с серверными API и транспортными протоколами.

Такое разделение соответствует принципу единственной ответственности (SRP) и позволяет локализовать изменения при модификации отдельных частей системы.

Приложение разработано в гибридной мобильной оболочке (Ionic + Angular), что обеспечивает:

- адаптивную вёрстку под различные размеры экранов;
- единый код для веб- и мобильных платформ;
- использование нативных возможностей устройства при необходимости.

4.7.2 Управление состоянием и реактивная модель

Обновление пользовательского интерфейса реализовано на основе реактивной модели программирования. Все изменения состояния представляются в виде потоков событий, которые распространяются по компонентам приложения.

Пусть S_t — состояние интерфейса в момент времени t, а E_t — событие (например, получение сообщения). Тогда переход состояния описывается как:

$$S_{t+1} = f(S_t, E_t),$$

где f — функция обновления состояния.

Источниками событий являются:

- входящие сообщения через WebSocket (SignalR);
- ответы HTTP API;
- пользовательские действия (ввод текста, отправка сообщения);
- внутренние события приложения (таймеры, изменения соединения).

Полученные данные сначала обрабатываются в слое состояния, после чего передаются в слой представления. В случае зашифрованных сообщений выполняется локальная расшифровка перед отображением.

Такой подход обеспечивает:

- слабую связанность компонентов;
- предсказуемость обновлений интерфейса;
- устойчивость к частичным сбоям (например, при ошибках расшифрования).

4.7.3 Интеграция с транспортным уровнем

Клиент взаимодействует с сервером по двум каналам:

- HTTP API — для выполнения запросов и получения истории данных;
- WebSocket (SignalR) — для доставки событий в реальном времени.

Сетевое взаимодействие инкапсулировано в сервисном слое, что позволяет:

- централизовать обработку ошибок;
- реализовать повторные попытки (retry);
- изолировать транспортную логику от UI-компонентов.

4.7.4 Конфигурация окружения

Параметры окружения (URL API, настройки WebSocket, режимы работы) выносятся в конфигурационные файлы сборки. Это позволяет:

- использовать различные окружения (development, staging, production);
- изменять параметры без модификации исходного кода;
- упростить процесс развертывания.

Конфигурация подставляется на этапе сборки приложения и доступна через сервис конфигурации.

4.7.5 Обработка состояний и ошибок

Пользовательский интерфейс учитывает различные состояния взаимодействия с сервером:

- загрузка данных (loading);
- успешное выполнение операции (success);
- ошибка (error);
- отсутствие соединения (offline).

Особое внимание уделяется различию между:

- транзитными ошибками (например, потеря соединения);
- ошибками валидации (ответы с кодами $4xx$);
- внутренними ошибками сервера ($5xx$).

Это позволяет корректно отображать сообщения пользователю и выбирать стратегию повторного запроса.

4.7.6 Согласованность с серверной логикой

Пользовательский интерфейс синхронизирован с логикой backend-сервиса. В частности:

- повторные запросы выполняются идемпотентно;
- состояние сообщений согласуется с серверной историей;
- визуализация статусов соответствует жизненному циклу данных.

Такой подход обеспечивает корректное поведение интерфейса при повторных попытках выполнения операций и при частичных сбоях сети.

4.7.7 Преимущества архитектуры UI

Применённая архитектура клиентского приложения обеспечивает:

- масштабируемость интерфейса;
- повторное использование компонентов;
- устойчивость к ошибкам;
- удобство сопровождения и расширения функциональности.

Таким образом, пользовательский интерфейс является не просто визуальным слоем, а полноценной частью распределённой системы, взаимодействующей с серверной логикой и обеспечивающей корректную обработку пользовательских сценариев.

4.8 Тестирование

Проверка работоспособности системы организована по многоуровневой схеме, охватывающей как отдельные компоненты, так и их взаимодействие в рамках полного вычислительного контура. Такой подход соответствует принципу пирамиды тестирования и позволяет обеспечить баланс между стоимостью сопровождения тестов и полнотой покрытия функциональности.

В рамках работы реализованы следующие уровни тестирования:

- модульные тесты бизнес-логики и криптографических примитивов;
- интеграционные тесты HTTP API;
- офлайн-оценка качества RAG-пайплайна;
- нагрузочное тестирование канала реального времени (SignalR).

Использованные методики, артефакты и полученные численные результаты достаточны для воспроизведения выводов в рамках одного тестового стенда. Численные оценки RAG воспроизведены из ноутбука `evaluation_bootstrap.ipynb` (каталог `RAG-test/OpenAITest`); визуализации соответствуют файлам `retrieve-eval.png` и `gen-eval.png`.

4.8.1 Модульные и интеграционные тесты API

Модульные тесты реализованы на платформе `xUnit` и направлены на проверку устойчивых к регрессии компонентов системы без поднятия сетевого стека. В частности, тестируются:

- корректность алгоритмов хеширования паролей;
- преобразование и валидация входных данных;
- отдельные элементы бизнес-логики.

Интеграционные тесты выполняются с использованием `WebApplicationFactory`, предоставляющей тестовый хост ASP.NET Core. В тестовом окружении:

- используется изолированная in-memory база данных Entity Framework;
- фиксируются параметры аутентификации (JWT);
- отключаются внешние зависимости.

Такая конфигурация позволяет воспроизводить сквозные сценарии:

- регистрация и аутентификация пользователей;
- работа с API интервью;
- загрузка и обработка медиа;
- взаимодействие с базой данных.

Дополнительно предусмотрены тесты, выполняемые в условиях, приближенных к production:

- взаимодействие с контейнеризованным API;
- интеграция с внешним ASR-провайдером при наличии секретов.

Данный набор тестов обеспечивает регрессионный контроль REST-контракта и согласованности с хранилищем при минимальной стоимости поддержки.

4.8.2 Непрерывная интеграция и окружения

Тестирование интегрировано в конвейер непрерывной интеграции (CI). Проверки, зависящие от внешних сервисов, выполняются условно — при наличии соответствующих переменных окружения.

Это позволяет разделить тесты на:

- базовые (выполняются при каждом коммите);
- расширенные (выполняются в ночных или ручных прогонах).

Такой подход снижает вероятность ложных отказов при отсутствии внешних зависимостей и обеспечивает воспроизводимость базового тестового контура.

4.8.3 Оценка качества RAG (офлайн)

Качество подсистемы RAG оценивалось в офлайн-режиме с использованием метода *LLM-as-judge*. Для каждой метрики строилась бутстрап-оценка среднего значения и двустороннего 95 % доверительного интервала.

Этап извлечения (retrieval) На этапе извлечения оценивались:

- релевантность извлечённых чанков запросу;
- согласованность с эталонным текстом;
- разность метрик как индикатор избыточной релевантности.

Результаты (табл. 1 и рис. 7) показывают, что модель `text-embedding-3-large` демонстрирует наилучшие показатели по всем ключевым метрикам. При этом наблюдается устойчивый разрыв между релевантностью и согласованностью, что указывает на склонность retrieval возвращать потенциально релевантные, но не всегда строго подтверждённые эталоном фрагменты.

Таблица 1: Chunk-judge (retrieval): среднее и 95 % ДИ бутстрапа (N=10 000), оценщик `gpt-4o-mini`. Источник: `evaluation_bootstrap.ipynb`.

Модель вложений	Судья	Релевантность запросу	Согласованность с эталоном	Релевантность − согласованность
`text-embedding-3-large`	`gpt-4o-mini`	4,766 [4,708; 4,821]	3,694 [3,584; 3,803]	1,072 [0,974; 1,171]
`text-embedding-3-small`	`gpt-4o-mini`	4,540 [4,460; 4,616]	3,477 [3,364; 3,592]	1,064 [0,971; 1,156]
`text-embedding-ada-002`	`gpt-4o-mini`	4,503 [4,419; 4,584]	3,434 [3,312; 3,555]	1,069 [0,974; 1,165]

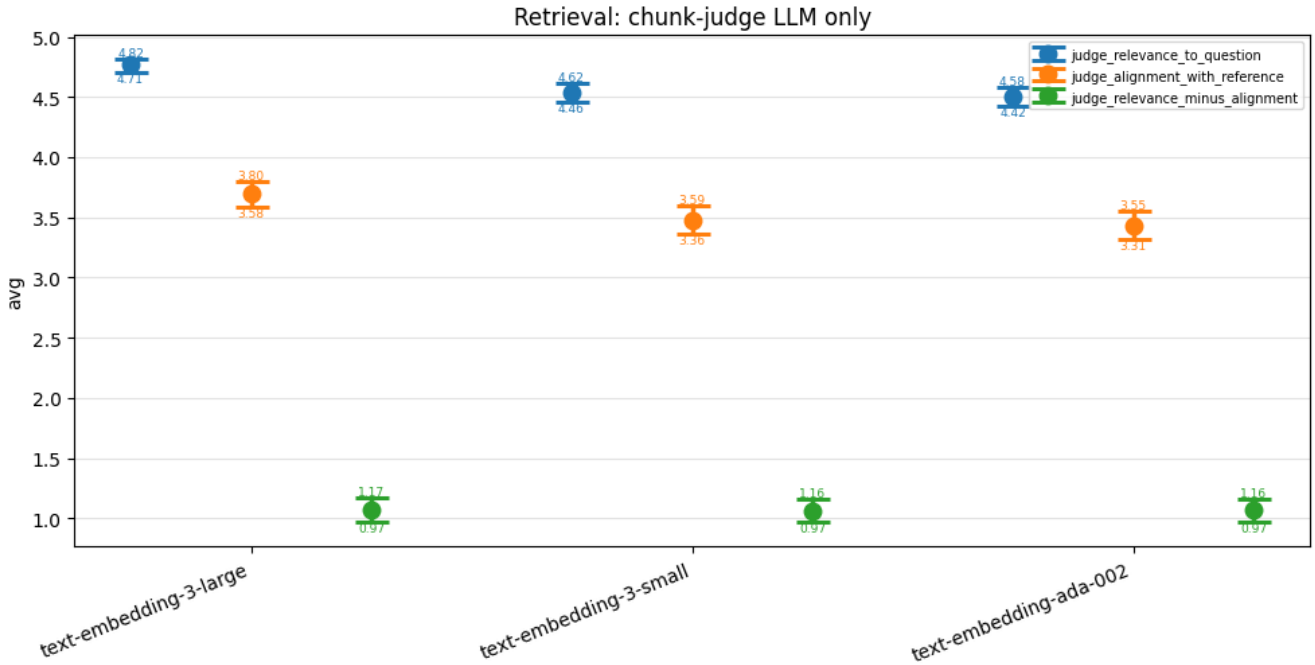

Рис. 7: Визуализация метрик chunk-judge по трём моделям вложений (те же данные, что в табл. 1).

Этап генерации На этапе генерации оценивались:

- соблюдение инструкции;
- опора на переданные чанки;
- корректность относительно контекста;
- соответствие эталонному ответу.

Таблица 2: Answer-judge (генерация): среднее и 95 % ДИ бутстрапа, судья `gpt-4o`, фиксированный retrieval (`text-embedding-3-large`, top-5). Источник: `evaluation_bootstrap.ipynb`.

Генеративная модель	n	Соблюдение инструкции	Верность чанкам	Корректность по чанкам	Соответствие эталону
`gpt-4o-mini`	346	3,864 [3,754; 3,971]	3,558 [3,436; 3,676]	3,558 [3,436; 3,679]	3,020 [2,887; 3,150]
`gpt-5.4`	346	4,272 [4,165; 4,373]	3,971 [3,853; 4,090]	3,974 [3,855; 4,092]	3,303 [3,185; 3,419]

Сравнение моделей (табл. 2 и рис. 8) показывает, что модель `gpt-5.4` демонстрирует более высокие значения по всем метрикам, особенно по показателю соответствия эталону. Это подтверждает влияние качества генеративной модели при фиксированном retrieval.

Интерпретация результатов Полученные результаты позволяют сделать следующие выводы:

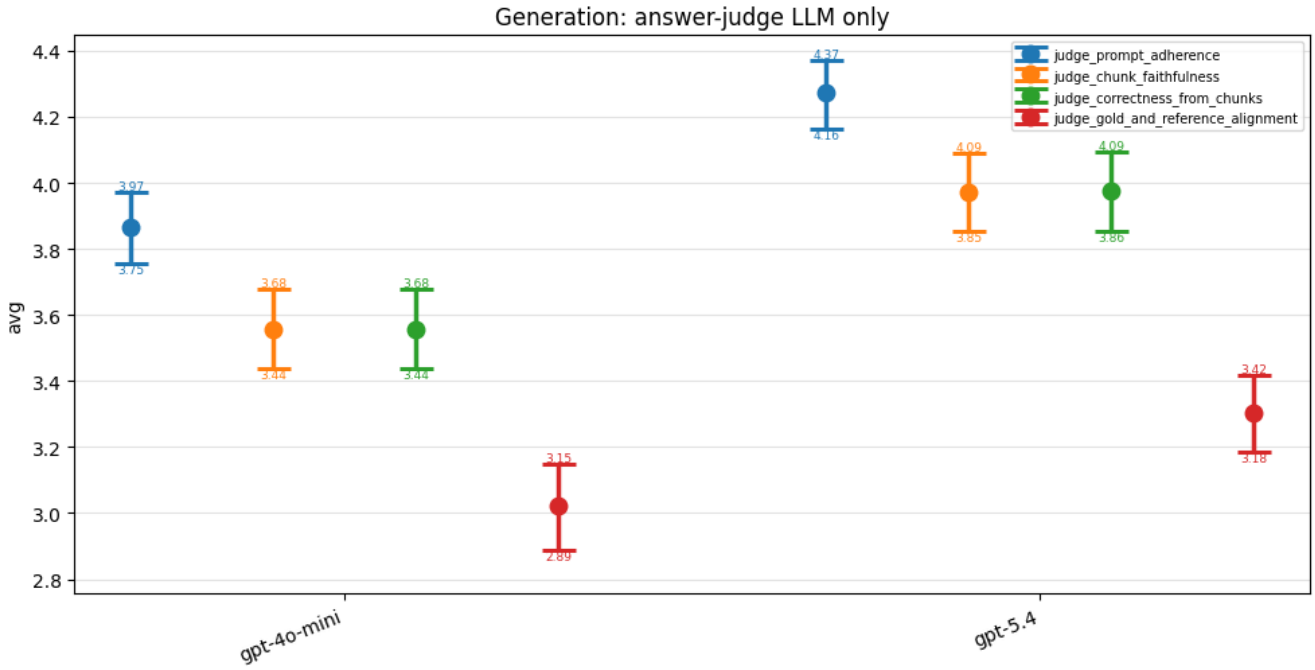

Рис. 8: Визуализация метрик answer-judge для двух генеративных моделей (те же данные, что в табл. 2).

- качество retrieval является критическим фактором для итогового качества ответа;
- улучшение генеративной модели повышает согласованность и точность;
- существует компромисс между вычислительной стоимостью и качеством результата.

4.8.4 Нагрузочное тестирование канала SignalR

Для оценки устойчивости канала реального времени использован набор нагрузочных сценариев на базе клиента `@microsoft/signalr`. Тестирование выполнялось на изолированном стенде без записи в базу данных, что позволило измерить чистую пропускную способность транспортного уровня. Сводка численных результатов приведена в табл. 3; иллюстрации — в рис. 9–11.

Сценарий `flood` Сценарий моделирует интенсивную нагрузку:

$$100 \text{ клиентов} \times 20 \text{ сообщений/с} = 2000 \text{ сообщений/с}.$$

В ходе теста зафиксировано:

- 59 640 успешных сообщений;
- средняя пропускная способность $\approx$ 1988 сообщений/с;
- $p_{50} \approx 6{,}07$ мс;
- $p_{95} \approx 10{,}69$ мс;
- $p_{99} \approx 15{,}71$ мс;
- нарушений порядка доставки не обнаружено.

Полученные значения близки к теоретическому пределу, что указывает на отсутствие узких мест в обработке сообщений.

Сценарий `ping-series` Сценарий измеряет круговую задержку (RTT):

- 120 измерений;
- медиана 3,90 мс;
- p_{95} 5,86 мс;
- максимум 13,92 мс;
- ошибок не зафиксировано.

Низкая вариативность задержек свидетельствует о стабильной работе канала.

Сценарий `hold` Сценарий проверяет устойчивость при длительном удержании соединений:

- 500 одновременных соединений;
- длительность 600 с;
- неожиданных разрывов: 0;
- переподключений: 0.

Результаты подтверждают устойчивость системы при длительной нагрузке.

Таблица 3: Сводка нагрузочных сценариев SignalR (скриншоты консоли: `figures/loadtest-*.png`).

Сценарий	Параметры запуска	Зафиксированные метрики
`flood`	100 клиентов, 20 msg/s на клиента, 30 с, `parallel=20`	59 640 вызовов; пропускная способность ≈ 1988 msg/s; p_{50}≈6,07 мс, p_{95}≈10,69 мс, p_{99}≈15,71 мс; нарушений порядка 0
`ping-series`	120 с, интервал 1000 мс	120 измерений RTT, сбоев 0; min 0,72 мс, p_{50} 3,90 мс, p_{95} 5,86 мс, max 13,92 мс
`hold`	500 соединений, 600 с idle	Все 500 подключены; неожиданных разрывов 0; переподключений 0

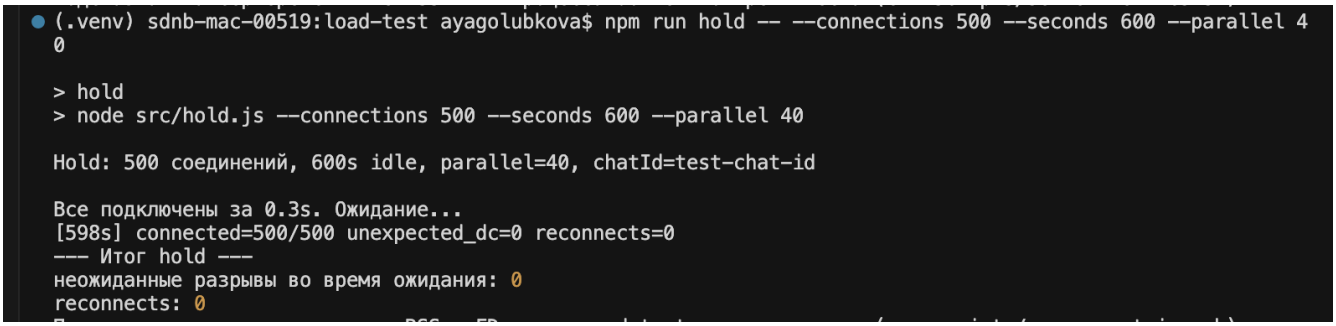

Рис. 9: Консольный вывод сценария `flood`: пропускная способность и перцентили задержки.

4.8.5 Выводы по результатам тестирования

Проведённое тестирование показывает, что система:

- корректно реализует бизнес-логику и API;
- демонстрирует устойчивую работу при интеграции компонентов;
- обеспечивает приемлемое качество RAG-пайплайна;
- выдерживает высокую нагрузку в канале реального времени.

Таким образом, реализованная архитектура соответствует требованиям производительности, масштабируемости и надёжности.

```
● (.venv) sdnb-mac-00519:load-test ayagolubkova$ npm run ping-series -- --seconds 120 --interval-ms 1000

  > ping-series
  > node src/ping-series.js --seconds 120 --interval-ms 1000


  --- Итог ping-series ---
  samples: 120 failures: 0
  RTT ms — min: 0.72 p50: 3.90 p95: 5.86 max: 13.92
```

Рис. 10: Консольный вывод сценария `ping-series`: статистика RTT.

```
● (.venv) sdnb-mac-00519:load-test ayagolubkova$ npm run flood -- --clients 100 --msg-per-sec-per-client 20 -
  -seconds 30 --parallel 20

  > flood
  > node src/flood.js --clients 100 --msg-per-sec-per-client 20 --seconds 30 --parallel 20

  Flood: 100 клиентов × 20 msg/s × 30s, chatId=test-chat-id, ramp parallel=20

  --- Итог flood ---
  invoke+delivery samples: 59640
  LoadTestBroadcast вызовов (sent): 59640
  throughput delivered/sec: 1988.0
  latency ms — p50: 6.07 p95: 10.69 p99: 15.71
  order violations: 0
```

Рис. 11: Консольный вывод сценария `hold`: удержание 500 соединений.

Список литературы

[1] AssemblyAI. *AssemblyAI: API Reference*, 2024. Справочник по эндпоинтам и параметрам транскрипции. URL: https://www.assemblyai.com/docs/api-reference.

[2] AssemblyAI. *AssemblyAI: Documentation*, 2024. Общая документация: обзор API, модели распознавания речи, руководства по интеграции. URL: https://www.assemblyai.com/docs.

[3] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer, Berlin, 2002.

[4] Yiran Deng, Ryan Mahan, Wenxuan Lin, Benjamin Van Durme, and Yulia Kim. Prompt rewriting for improving retrieval-augmented generation. arXiv preprint arXiv:2311.02897, 2023. URL: https://arxiv.org/abs/2311.02897.

[5] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. *Transactions of the Association for Computational Linguistics (TACL)*, 9:874–888, 2021. URL: https://arxiv.org/abs/2007.01282.

[6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020. URL: https://arxiv.org/abs/2004.04906.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL: https://arxiv.org/abs/2005.11401.

[8] Jimmy Lin and Xinyu Ma. A few brief notes on deep impact, COIL, and neural ranking models. arXiv preprint arXiv:2102.04906, 2021. URL: https://arxiv.org/abs/2102.04906.

[9] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. arXiv preprint arXiv:2303.16634, 2023. URL: https://arxiv.org/abs/2303.16634.

[10] Xinbei Ma, Yeyun Gong, Jing He, Hai Zhao, Nan Duan, and Chen Wang. Query rewriting for retrieval-augmented large language models. arXiv preprint arXiv:2305.14328, 2023. URL: https://arxiv.org/abs/2305.14328.

[11] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008. URL: https://nlp.stanford.edu/IR-book/.

[12] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996.

[13] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv preprint arXiv:1901.04085, 2019. URL: https://arxiv.org/abs/1901.04085.

[14] Qdrant. *Qdrant Vector Database: High-Performance Vector Search Engine*, 2024. Официальная документация. URL: https://qdrant.tech/documentation/.

[15] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[16] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[17] Kurt Shuster, Jing Xu, Melanie Johnson, Arthur Szlam, Arthur Rolland, and Y-Lan Boureau. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2021. URL: https://arxiv.org/abs/2104.07567.

[18] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2021. URL: https://arxiv.org/abs/2104.08663.

[19] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Zhuohan Li, Zhanghao Zhuang, Yonghao Zhuang, Joseph E. Gonzalez Zhu, and Ion Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. arXiv preprint arXiv:2403.04132, 2024. URL: https://arxiv.org/abs/2403.04132.

[20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. arXiv preprint arXiv:2306.05685, 2023. URL: https://arxiv.org/abs/2306.05685.